Topic: Customer Support Agentic RAG System Using Azure OpenAI

Abstract

Customer support teams often face challenges in efficiently responding to customer queries due to the large volume of historical tickets, knowledge base articles, and external information sources. Traditional search-based systems struggle to provide context-aware and intelligent responses. To address this challenge, an Agentic Retrieval-Augmented Generation (RAG) system was developed using Azure OpenAI, Azure Embeddings, FAISS Vector Database, Cohere Reranking, Wikipedia Search, and Web Search tools.

The system combines semantic search, keyword search, reranking, and intelligent tool selection to provide accurate and context-aware responses to customer support queries. Historical support tickets are converted into vector embeddings and stored in a FAISS vector database. When a user submits a query, the system retrieves relevant support records, reranks them using Cohere, and generates responses using Azure OpenAI GPT-5 Mini. Additionally, the agent can dynamically select external tools such as Wikipedia and Web Search whenever information is unavailable in the internal knowledge base. A FastAPI backend and Streamlit frontend provide an interactive user interface for real-time customer support assistance.

1. Introduction

Modern organizations receive thousands of customer support requests covering technical issues, billing concerns, account management problems, and general inquiries. Support teams often spend significant time searching through previous tickets and documentation to identify solutions.

Large Language Models (LLMs) have improved conversational AI capabilities, but they suffer from knowledge limitations and potential hallucinations. Retrieval-Augmented Generation (RAG) addresses this issue by grounding responses in external knowledge sources.

This project develops an Agentic RAG system capable of:

- Retrieving relevant customer support tickets.
- Understanding customer intent.
- Selecting appropriate tools dynamically.
- Providing grounded and context-aware answers.
- Leveraging external information when internal knowledge is insufficient.

2. Problem Statement

Customer support organizations face several challenges:

- Large volumes of historical support tickets.
- Difficulty locating relevant past resolutions.
- Delayed response times.
- Inconsistent support quality.
- Inability of traditional chatbots to access external sources.

- Risk of hallucinated responses by standalone LLMs.

The objective is to build an intelligent support assistant capable of retrieving relevant information from historical support data and external sources while providing accurate, context-aware, and explainable responses.

3. Proposed Solution

An Agentic Retrieval-Augmented Generation architecture was developed consisting of multiple components:

Internal Knowledge Retrieval

- Historical customer support tickets stored in CSV format.
- Documents converted into semantic embeddings.
- Embeddings stored in FAISS Vector Database.

Hybrid Search Strategy

The solution combines:

- Semantic Search (FAISS)
- Keyword Search (BM25)

This improves retrieval accuracy by leveraging both semantic meaning and exact keyword matching.

Reranking Layer

Retrieved documents are reranked using Cohere Rerank to ensure the most relevant ticket information is prioritized before sending context to the language model.

Agentic Decision Making

GPT-5 Mini acts as an intelligent agent and dynamically selects among:

1. Customer Support RAG Tool
2. Wikipedia Search Tool
3. Web Search Tool

Response Generation

GPT-5 Mini generates the final response based on:

- Retrieved ticket context
- External search information
- Conversation history
-

4. Methodology

The development of the Customer Support Agentic RAG System followed a structured pipeline consisting of data ingestion, vectorization, retrieval, reranking, agent orchestration, and response generation.

Phase 1: Data Collection and Preparation

A customer support ticket dataset was used as the primary knowledge source. The dataset contained ticket information such as Ticket ID, Subject, Description, Issue Category, Priority Level, Support Channel, Resolution Time, Assigned Agent, and Customer Satisfaction Score.

The dataset was loaded using LangChain's CSV Loader and converted into document objects suitable for retrieval. To improve retrieval efficiency, the textual content was preprocessed and split into manageable chunks using Recursive Character Text Splitter.

Phase 2: Embedding Generation

After preprocessing, Azure OpenAI Embeddings were used to transform each document chunk into high-dimensional vector representations.

The embedding process converts textual information into numerical vectors that capture semantic meaning rather than exact keyword matches. This enables the system to understand user intent and retrieve contextually similar support tickets.

Phase 3: Vector Database Construction

The generated embeddings were stored in a FAISS (Facebook AI Similarity Search) vector database.

FAISS enables efficient similarity search by indexing the document vectors and retrieving the most relevant support tickets when a user submits a query. The vector store was persisted locally and reused during inference, eliminating the need to regenerate embeddings repeatedly.

Phase 4: Hybrid Retrieval System

To improve retrieval quality, a hybrid retrieval approach was implemented.

The retrieval layer combines:

- Semantic Search using FAISS
- Keyword Search using BM25

Both retrieval methods are executed simultaneously and combined through an Ensemble Retriever. FAISS identifies semantically relevant documents, while BM25 captures exact keywords and domain-specific terminology.

This hybrid retrieval strategy improves accuracy and reduces the likelihood of missing important information.

Phase 5: Contextual Reranking

The initially retrieved documents may contain partially relevant results. To improve precision, Cohere Rerank was integrated into the retrieval pipeline.

The reranking layer evaluates the retrieved documents and reorders them according to their relevance to the user query. Only the highest-ranking documents are passed to the language model.

Benefits of reranking include:

- Improved contextual relevance
- Reduced noise in retrieved results
- Higher answer accuracy
- Better response grounding

Phase 6: Agentic Tool Orchestration

An Agentic AI architecture was implemented to enable dynamic tool selection.

The system provides three tools:

Customer Support RAG Tool

- Searches historical support tickets
- Handles account issues, billing issues, refunds, login problems, and technical support requests

Wikipedia Tool

- Retrieves factual and educational information
- Supports definition-based and conceptual queries

Web Search Tool

- Retrieves real-time and external information
- Handles recent updates, product releases, and publicly available documentation

GPT-5 Mini acts as the reasoning engine and determines which tool should be used based on the user's query.

Phase 7: Response Generation

After retrieving the most relevant information, GPT-5 Mini generates the final response.

The model receives:

- User query
- Retrieved support-ticket context
- Previous conversation history
- External information (if applicable)

The generated response is context-aware, grounded in retrieved information, and tailored to customer-support scenarios.

Phase 8: Application Deployment

The final solution was deployed using:

FastAPI

- REST API endpoint creation
- Agent management
- Query processing

Streamlit

- Interactive user interface
- Query submission
- Response visualization
- Conversation history management

The complete system provides a real-time conversational experience for end users.

5. Results

The developed Customer Support Agentic RAG System successfully demonstrated the ability to retrieve, analyze, and generate context-aware responses for customer-support scenarios.

Retrieval Performance

The integration of Azure OpenAI Embeddings with FAISS enabled efficient semantic retrieval of historical support tickets. The system successfully identified relevant tickets even when user queries did not contain exact keyword matches.

Hybrid Search Improvement

The combination of FAISS semantic retrieval and BM25 keyword retrieval improved overall search effectiveness. Compared to using semantic search alone, the hybrid retrieval mechanism provided more robust results for both technical and business-related queries.

Cohere Reranking Effectiveness

The Cohere Rerank model significantly improved document relevance by prioritizing the most contextually appropriate support tickets before response generation. This reduced irrelevant context and improved the quality of generated answers.

Agent Tool Selection

The agent successfully selected among multiple tools based on query intent:

Query Type	Tool Selected
Billing, Account, Refund Issues	Customer Support RAG
Definitions and Concepts	Wikipedia
Latest Updates and External Information	Web Search

This demonstrated the capability of the agent to dynamically route requests to the most appropriate knowledge source.

User Experience

The Streamlit interface provided an intuitive conversational experience, while FastAPI ensured low-latency communication between the frontend and backend components.

Key Achievements

- Successful implementation of Agentic RAG architecture.
- Efficient retrieval of historical support tickets.
- Reduced hallucinations through context-grounded responses.
- Dynamic tool selection based on query intent.
- Improved answer quality through Cohere reranking.
- Real-time interaction through a web-based interface.

10. Conclusion

This project successfully developed an Agentic Retrieval-Augmented Generation (RAG) system for customer support automation using Azure OpenAI, Azure Embeddings, FAISS, LangChain, Cohere Reranking, Wikipedia Search, and Web Search integration.

The solution effectively combines semantic retrieval, keyword search, document reranking, and intelligent agent-based tool selection to provide accurate and contextually relevant responses. By grounding responses in historical support data and external knowledge sources, the system minimizes hallucinations while significantly improving answer reliability.

The implementation demonstrates how modern AI systems can enhance customer-support operations by reducing response times, improving consistency, and enabling intelligent knowledge discovery across multiple information sources.

The developed architecture is scalable, modular, and suitable for enterprise deployment. Future enhancements may include multilingual support, CRM integration, ticket-priority prediction, sentiment analysis, real-time monitoring, and advanced multi-agent orchestration using LangGraph.

Overall, the project showcases the practical application of Agentic AI and Retrieval-Augmented Generation in solving real-world customer-support challenges while delivering an efficient and user-friendly conversational experience.